

Prompt Engineering Experiments

A hands-on report exploring five core prompting techniques — Role Prompting, Chain-of-Thought, Few-Shot, Structured JSON Output, and Prompt Optimization — run against a Claude LLM, with prompts, real outputs, observations, and improvements for each.

Experiment 1 — Role Prompting

PROMPT

You are a senior computer vision engineer at an autonomous vehicle company with 10 years of experience in real-time object detection. In 4 concise bullet points aimed at a technical ML audience, explain how to improve a YOLOv8 model's precision on small objects.

OUTPUT

- Increase input resolution or use tiled/sliced inference (e.g. SAHI) so small objects occupy more pixels before detection.
- Add a high-resolution P2 detection head to the FPN so shallow, fine-grained feature maps are used for small-object anchors.
- Apply mosaic and copy-paste augmentation, oversampling images with small objects to correct scale imbalance in training data.
- Reweight the loss (e.g. Focal Loss with higher small-object weight) and re-tune anchor/scale assignment to cut false negatives.

OBSERVATIONS

Assigning a role ("senior CV engineer, 10 years experience") shifted the tone from generic advice to specific, jargon-appropriate recommendations (SAHI, FPN, P2 head, Focal Loss) that a beginner-framed prompt rarely produces on its own. The role also implicitly set the audience and depth — no separate instruction was needed for "technical."

IMPROVEMENTS

Pin down the target framework/version (e.g. "YOLOv8n vs YOLOv8x") and ask the model to rank the 4 techniques by expected impact-to-effort ratio, so the output is directly actionable rather than just a list of valid options.

Experiment 2 — Chain-of-Thought Prompting

PROMPT

A freelancer charges \$25/hour. He works 6 hours on Monday and 4.5 hours on Tuesday. The platform takes a 15% service fee from his total earnings. Solve step by step: how much does he take home?

OUTPUT

Step 1: Total hours worked = $6 + 4.5 = 10.5$ hours
Step 2: Gross earnings = $10.5 \times \$25 = \262.50

Step 3: Platform fee = 15% of \$262.50 = $0.15 \times 262.50 = \$39.375$
Step 4: Take-home pay = $\$262.50 - \$39.375 = \$223.125 \approx \223.13

OBSERVATIONS

Explicitly asking the model to "solve step by step" forced it to externalize intermediate arithmetic (hours → gross → fee → net) instead of jumping straight to a final number. This is the same multi-step billing math a freelance-invoicing feature would need, and the visible steps make it trivial to spot-check for errors — the same prompt without "step by step" tends to skip straight to a final figure with no audit trail.

IMPROVEMENTS

Add "double-check your arithmetic in a final step before giving the answer" to catch self-inconsistencies, and request the answer rounded to 2 decimal places explicitly, since currency values shouldn't carry 3+ decimal digits.

Experiment 3 — Few-Shot Prompting

PROMPT

Classify the tone of freelance client reviews as Positive, Neutral, or Negative.

Review: "Delivered exactly what I asked, ahead of schedule." -> Positive

Review: "Work was okay, took a bit longer than expected." -> Neutral

Review: "Missed two deadlines and barely communicated." -> Negative

Review: "Great communication throughout, but the final design needed two rounds of revisions before it matched the brief." -> ?

OUTPUT

Neutral

Reasoning: the review has one clearly positive signal (communication) offset by one mildly negative signal (extra revision rounds), landing it between the "Positive" and "Negative" anchor examples rather than matching either extreme.

OBSERVATIONS

The three labeled examples anchored both the output format (single-word label) and the calibration of "Neutral" — without them, a zero-shot prompt is more likely to default to a binary Positive/Negative call and miss mixed-sentiment cases. Few-shot examples act as a mini rubric, not just formatting hints.

IMPROVEMENTS

Add one more example that is explicitly mixed/neutral (closer to the real edge case) rather than three examples that sit at the extremes, and constrain the output to the label only when the tool feeding this into a dashboard doesn't need the reasoning text.

Experiment 4 — Structured Output (JSON)

PROMPT

Extract the following text into JSON with keys: name, role, skills (array), years_experience, location. Respond with only valid JSON, no explanation.

Text: "Waqar Haider is an AI Engineer and Full Stack Developer based in Faisalabad, Pakistan, with experience in YOLOv8, FastAPI, and Next.js, having worked on applied AI projects for about 2 years."

OUTPUT

```
{
  "name": "Waqar Haider",
  "role": "AI Engineer and Full Stack Developer",
  "skills": ["YOLOv8", "FastAPI", "Next.js"],
  "years_experience": 2,
  "location": "Faisalabad, Pakistan"
}
```

OBSERVATIONS

Specifying the exact key names and types (skills as an array, years_experience as a number) up front produced clean, directly parseable JSON with no markdown code fences or commentary wrapped around it — the "respond with only valid JSON" instruction was necessary; without it, the model tends to add a sentence of preamble before the object.

IMPROVEMENTS

For production use, pair this prompt with a JSON Schema (via the API's structured-output / tool-use mode rather than plain text instructions) and validate the response server-side, since prose-based JSON instructions can still occasionally drift on edge cases like missing fields or ambiguous data.

Experiment 5 — Prompt Optimization

VERSION 1 (VAGUE)

Write about my project.

Generic, unusable output: the model has no project details, audience, tone, or length target, so it either asks a clarifying question or produces a placeholder paragraph about "a project" with no real content.

VERSION 2 (ADDED CONTEXT)

Write a short paragraph describing my final year project, MediScan, a YOLOv8-based bone fracture detection system with 89.4% mAP@0.5.

Usable but generic in tone – reads like a textbook description, no clear audience, and length is unpredictable (could be 40 words or 200).

VERSION 3 (OPTIMIZED: ROLE + AUDIENCE + FORMAT + CONSTRAINT)

You are a technical writer. In 60-80 words, write a portfolio project description for "MediScan," a YOLOv8-based bone fracture detection system (89.4% mAP@0.5 on the GRAZPEDWRI-DX dataset). Audience: recruiters skimming a portfolio site. Tone: confident, concise, results-first.

MediScan is an AI-powered bone fracture detection system built on YOLOv8, achieving 89.4% mAP@0.5 on the GRAZPEDWRI-DX pediatric X-ray dataset. Designed to assist radiologists with faster, more consistent diagnoses, the model was trained end-to-end with custom data augmentation and evaluated using industry-standard object detection metrics. The project demonstrates practical deep learning deployment - from dataset curation and model training to Grad-CAM-based explainability - for a real clinical use case.

OBSERVATIONS

Each added constraint measurably improved output quality: context (V2) fixed factual accuracy, while role + audience + length + tone (V3) fixed the *usability* problem — V3 is close to drop-in-ready for a portfolio site, while V1 and V2 both need manual rewriting.

IMPROVEMENTS

Add a negative constraint ("avoid buzzwords like 'leveraging' and 'cutting-edge'") and a call-to-action instruction (e.g. "end with one sentence linking impact to a business/clinical outcome") to push V3 from good to portfolio-ready without a manual editing pass.

SUMMARY

Technique	Key Lever	Effect on Output
Role Prompting	Persona + expertise level	Domain-specific vocabulary, correctly calibrated depth
Chain-of-Thought	"Step by step" instruction	Visible reasoning, fewer arithmetic errors, easier to audit
Few-Shot	Labeled examples	Consistent format and calibrated judgment on edge cases
Structured JSON	Explicit schema + "only JSON"	Clean, parseable output with no extra text
Prompt Optimization	Role + audience + format + constraints	Iteratively raised output from unusable to near-final